

# DIPLOMARBEIT

## Zero-Emission-Challenge-Timingsystem



**Ausgeführt von:**

David Darnhofer

**Jahrgang/Klasse:**

2025/26/5AHIT

**Betreuer:**

DI Mock Hannes

**Projektpartner:**

HTL-Weiz

DI Purkarthofer Gott-  
fried

Direktor

DI Blattner Heimo

Abteilungsvorstand

---

**Abgabevermerk:**

Datum:

Unterschrift:

## 1 Eidesstattliche Erklärung

Ich erkläre an Eides statt, dass ich die vorliegende Diplomarbeit selbstständig und ohne fremde Hilfe verfasst, andere als die angegebenen Quellen und Hilfsmittel nicht benutzt und die den benutzten Quellen wörtlich und inhaltlich entnommenen Stellen als solche erkenntlich gemacht habe.

KI wurde ausschließlich unterstützend eingesetzt, zum Beispiel zur Recherche, zur sprachlichen Überarbeitung, zur Strukturierung oder zur Ideenfindung. Alle daraus entstandenen Inhalte wurden von mir selbst geprüft, überarbeitet und fachlich nachvollzogen.

Weiz, am 07. April 2026 David Darnhofer:

.....

## 2 Kurzbeschreibung

Diese Diplomarbeit befasst sich mit dem Konzept und der Implementierung eines Zeitmesssystems für die Zero Emission Challenge (ZEC), einem Kartwettbewerb der HTL Weiz. Ausgangspunkt war ein bestehendes System, welches allerdings schwer wartbar, kaum erweiterbar und mit einem aufwendigen Setup-Prozess verbunden war. Deshalb wurde eine komplette Neuentwicklung des Systems angestrebt. Das Ziel war eine Lösung, die den gesamten Ablauf von der Zeitmessung über die Punktauswertung bis zur Anzeige der Ergebnisse sowie die Team- und Benutzerverwaltung abdeckt. Weiters sollte das System eine hohe Erweiterbarkeit, unkomplizierte Wartbarkeit und eine einfache Inbetriebnahme gewährleisten. Die Hauptkomponenten dieses Systems umfassen einen Backend-Server zur Datenverwaltung und -verarbeitung, eine Webanwendung zur Erfassung von Zeitstempeln, welche diese per MQTT-Protokoll von Lichtschranken empfängt und an den Server weiterleitet, sowie eine weitere Webanwendung zur Darstellung der Ergebnisse und zur Verwaltung von Teams und Benutzern. Die Authentifizierung und Zugriffskontrolle werden mit Keycloak umgesetzt, während die Datenspeicherung in einer PostgreSQL-Datenbank erfolgt. Das System basiert auf einer Microservice-Architektur und kann in einem Kubernetes-Cluster betrieben werden. Zur Sicherstellung der Softwarequalität wurden automatisierte Continuous Integration / Continuous Deployment (CI/CD)-Pipelines erstellt, welche das Testen, Bauen und Veröffentlichen der Container-Images übernehmen.

## Abstract

This diploma thesis deals with the concept and implementation of a timing system for the Zero Emission Challenge (ZEC), a karting competition organized by HTL Weiz. The starting point was an existing system that was difficult to maintain, hardly expandable, and required a complex setup process. Therefore, a complete redevelopment of the system was pursued. The goal was a solution that covers the entire workflow, from time measurement and score calculation to result presentation, as well as team and user management. In addition, the system was designed to provide high extensibility, simple maintenance, and easy deployment. The main components of this system include a backend server for data management and processing, a web application for capturing timestamps, which receives them from light barriers via the MQTT protocol and forwards them to the server, and another web application for displaying results and managing teams and users. Authentication and access control are implemented with Keycloak, while data is stored in a PostgreSQL database. The system is based on a microservice architecture and can be operated in a Kubernetes cluster. To ensure software quality, automated CI/CD pipelines were created to handle testing, building, and publishing container images.

### 3 Vorwort

Das Thema der Diplomarbeit (DA) wurde von der HTL Weiz ausgegeben. Ich wurde von Herrn Prof. Mock auf diese Arbeit aufmerksam gemacht und er vermittelte mir diese Aufgabe. An dieser Stelle möchte ich mich herzlich bei ihm für die Betreuung und Unterstützung während der gesamten Diplomarbeit bedanken. Durch seine fachliche Expertise und seine Unterstützung konnte ich die Arbeit zielgerichtet und erfolgreich durchführen. Ebenso möchte ich meinen Freunden danken, die mich vor allem mit ihrer Faszination am Namen der DA (ZEC-Timing) immer wieder motiviert haben, weiterzumachen. Abschließend gilt der größte Dank meiner Familie. Sie hat mich mein ganzes Leben hinweg stets unterstützt und begleitet, wofür ich sehr dankbar bin. Ohne ihre Unterstützung wäre diese Arbeit nicht möglich gewesen.

Weiz, am 07. April 2026 David Darnhofer

# Inhalt

|          |                                                  |           |
|----------|--------------------------------------------------|-----------|
| <b>1</b> | <b>Eidesstattliche Erklärung</b>                 | <b>3</b>  |
| <b>2</b> | <b>Kurzbeschreibung</b>                          | <b>4</b>  |
| <b>3</b> | <b>Vorwort</b>                                   | <b>6</b>  |
| <b>4</b> | <b>Einleitung</b>                                | <b>9</b>  |
| 4.1      | Ziele . . . . .                                  | 9         |
| 4.2      | Individuelle Aufgabenstellung . . . . .          | 10        |
| <b>5</b> | <b>Theoretische Grundlagen</b>                   | <b>11</b> |
| 5.1      | Containerisierung . . . . .                      | 11        |
| 5.1.1    | Docker . . . . .                                 | 11        |
| 5.1.2    | Architektur von Docker . . . . .                 | 11        |
| 5.1.3    | Funktionsweise . . . . .                         | 12        |
| 5.1.4    | Docker Compose . . . . .                         | 12        |
| 5.1.5    | Vorteile der Containerisierung . . . . .         | 12        |
| 5.2      | Microservices . . . . .                          | 13        |
| 5.2.1    | Vorteile von Microservices . . . . .             | 13        |
| 5.2.2    | Nutzung von Microservices in dieser DA . . . . . | 13        |
| <b>6</b> | <b>Technologieauswahl</b>                        | <b>14</b> |
| 6.1      | GitHub . . . . .                                 | 14        |
| 6.2      | Nginx . . . . .                                  | 14        |
| 6.3      | Docker . . . . .                                 | 14        |
| 6.4      | PostgreSQL . . . . .                             | 15        |
| 6.5      | Keycloak . . . . .                               | 15        |
| 6.6      | SQLAlchemy . . . . .                             | 15        |
| 6.7      | FastAPI . . . . .                                | 16        |
| 6.8      | Next.js . . . . .                                | 16        |
| 6.9      | shadcn/ui . . . . .                              | 16        |
| <b>7</b> | <b>Systemarchitektur</b>                         | <b>17</b> |
| 7.1      | Überblick . . . . .                              | 17        |
| 7.2      | Server . . . . .                                 | 18        |
| 7.2.1    | Nginx API-Gateway . . . . .                      | 19        |
| 7.2.2    | Auth Service . . . . .                           | 20        |

|           |                                            |           |
|-----------|--------------------------------------------|-----------|
| 7.2.3     | User Service . . . . .                     | 20        |
| 7.2.4     | Team Service . . . . .                     | 21        |
| 7.2.5     | Challenge Service . . . . .                | 22        |
| 7.2.6     | Attempt Service . . . . .                  | 23        |
| 7.2.7     | Score Service . . . . .                    | 24        |
| 7.3       | Web App . . . . .                          | 25        |
| <b>8</b>  | <b>Implementation</b>                      | <b>28</b> |
| 8.1       | Server . . . . .                           | 28        |
| 8.1.1     | Zugriffsberechtigungen . . . . .           | 28        |
| 8.1.2     | API-Gateway . . . . .                      | 29        |
| 8.1.3     | CRUD-Operationen . . . . .                 | 31        |
| 8.1.4     | Authentifizierung mit Keycloak . . . . .   | 34        |
| 8.1.5     | Inter-Service-Kommunikation . . . . .      | 36        |
| 8.2       | Web App . . . . .                          | 38        |
| 8.2.1     | Kommunikation mit dem Server . . . . .     | 38        |
| 8.2.2     | Rollenbasierte Zugriffskontrolle . . . . . | 39        |
| <b>9</b>  | <b>Testen</b>                              | <b>40</b> |
| 9.1       | Teststrategie . . . . .                    | 40        |
| 9.2       | Pytest . . . . .                           | 40        |
| 9.2.1     | Unit-Tests . . . . .                       | 40        |
| 9.2.2     | Integrationstests . . . . .                | 41        |
| 9.3       | Playwright . . . . .                       | 41        |
| <b>10</b> | <b>Fazit und Ausblick</b>                  | <b>42</b> |
| 10.1      | Zusammenfassung . . . . .                  | 42        |
| 10.2      | Zukunftsausblick . . . . .                 | 42        |
| <b>11</b> | <b>Anhang</b>                              | <b>43</b> |
| <b>12</b> | <b>Verzeichnisse</b>                       | <b>47</b> |
| 12.1      | Abbildungsverzeichnis . . . . .            | 47        |
| 12.2      | Tabellenverzeichnis . . . . .              | 48        |
| 12.3      | Abkürzungsverzeichnis . . . . .            | 49        |
| 12.4      | Literaturverzeichnis . . . . .             | 50        |



## 4 Einleitung

### 4.1 Ziele

Das Ziel dieser Diplomarbeit ist es, ein wartbares, erweiterbares und einfach aufsetzbares Zeitmesssystem für die *Zero Emission Challenge* zu entwickeln, welches den gesamten Ablauf von der Zeiterfassung über die Punkteausswertung bis zur Ergebnisanzeige umfasst. Das System soll zusätzlich auch die Verwaltung von Teams und Benutzern ermöglichen und eine rollenbasierte Zugriffskontrolle implementieren.

Das entwickelte System soll:

- Renndaten aller Bewerbe zuverlässig erfassen, speichern und auswerten
- Versuchsdaten per Application Programming Interface (API) in das System speichern
- Eine webbasierte Plattform zur Verwaltung von Teams, Benutzern und Ergebnissen bereitstellen
- Sicherheit durch rollenbasierte Zugriffskontrolle und eine zentrale Authentifizierung über Keycloak gewährleisten
- Mittels Docker einfach und plattformunabhängig betreibbar sein
- Softwarequalität durch automatisierte Tests wie Pytest und Playwright sicherstellen

## 4.2 Individuelle Aufgabenstellung

Um diesen Zielen gerecht zu werden, wurden folgende Aufgaben definiert, die im Rahmen dieser Diplomarbeit umgesetzt werden sollen:

- Entwicklung eines in Microservices aufgeteilten RESTful-API-Backends zur Zeiterfassung, Punkteausswertung sowie Team- und Benutzerverwaltung
- Entwicklung einer webbasierten Anwendung mit unterschiedlichen Funktionen für die Benutzergruppen Administrator, Teamleiter und Zuschauer
- Implementierung eines dedizierten Authentifizierungsdienstes
- Rollenbasierte Zugriffskontrolle auf die Website sowie die API-Endpunkte
- Modellierung und Anbindung einer relationalen PostgreSQL-Datenbank zur Datenspeicherung
- Containerisierung aller Systemkomponenten mittels Docker
- Implementierung automatisierter Testfälle mit Pytest und Playwright

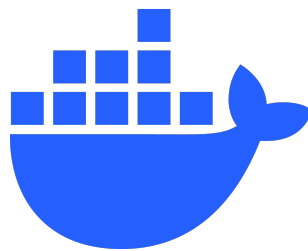
## 5 Theoretische Grundlagen

### 5.1 Containerisierung

Containerisierung ist eine Methode, bei der Anwendungen zusammen mit ihren Abhängigkeiten und Konfigurationsdateien in isolierten Umgebungen, sogenannten Containern, ausgeführt werden. Traditionell muss man, um eine Anwendung zu installieren, die zum jeweiligen Betriebssystem kompatible Version herunterladen. Mit Containerisierung ist es jedoch möglich, das gesamte Softwarepaket in einem Container auf verschiedenen Betriebssystemen auszuführen[1]

#### 5.1.1 Docker

Docker ist eine weitverbreitete, offene Plattform zur Erstellung und Verwaltung von Containern. Docker ermöglicht es, Anwendungen von der Infrastruktur zu trennen, indem es Container bereitstellt, die in jeder Umgebung konsistent laufen.[2]



**Bild 1:** Docker Logo

*Quelle:* <https://www.docker.com/company/newsroom/media-resources/>

#### 5.1.2 Architektur von Docker

Docker verwendet eine Client-Server-Architektur. Der *Docker-Client* kommuniziert dabei mit dem *Docker-Daemon* (`dockerd`), welcher die eigentliche Arbeit des Erstellens, Ausführens und Verteilens von Containern übernimmt. Die Kommunikation zwischen Client und Daemon erfolgt über eine REST-API, wahlweise über Unix-Sockets oder eine Netzwerkschnittstelle.[2]

### 5.1.3 Funktionsweise

Alles beginnt mit einem Dockerfile. Docker erstellt Images basierend auf den Anweisungen in einem Dockerfile. Ein Dockerfile ist eine Textdatei, die die Anweisungen enthält, wie der Quellcode gebaut wird, welche Pakete installiert werden müssen und wie die Anwendung gestartet wird. Docker-Images bestehen aus mehreren Schichten (Layers). Jede Anweisung im Dockerfile erzeugt eine neue Schicht, die auf den vorherigen aufbaut.[3]

Ein Container ist eine laufende Instanz eines Docker-Images. Er enthält alles, was die Anwendung benötigt, um zu laufen, einschließlich des Programmcodes. Docker-Container sind leichtgewichtig, da sie den Betriebssystemkernel des Host-Systems gemeinsam nutzen, anstatt wie virtuelle Maschinen ein ganzes Betriebssystem zu benötigen.[2]

### 5.1.4 Docker Compose

Docker Compose ist ein Werkzeug zur Definition und Verwaltung von Multi-Container-Applikationen. In einer einzigen YAML-Datei `compose.yaml` werden alle Komponenten einer Applikation beschrieben.[4] In dieser DA wird Docker Compose verwendet, um Backend-Services, Datenbanken, API-Gateway sowie den Authentifizierungsdienst mit einem einzigen Befehl starten zu können.

### 5.1.5 Vorteile der Containerisierung

Der Hauptvorteil der Containerisierung liegt in der Eliminierung des „*works on my machine*“-Problems. Ein Image, das einmal erstellt wurde, läuft auf jedem System identisch, unabhängig von der Infrastruktur, egal ob es sich um einen Raspberry Pi oder einen Server handelt. Da jeder Container komplett isoliert ist, eignet sich die Containerisierung auch sehr gut für die Microservice-Architektur, da jeder Service in einem eigenen Container laufen kann, ohne dass es zu Konflikten mit Abhängigkeiten anderer Services kommt. Das erleichtert sowohl die Entwicklung als auch das Deployment und den Betrieb, da das System immer identisch läuft. Weiters ermöglichen die Container eine leichte horizontale Skalierbarkeit. Mittels Orchestrierungstools wie Kubernetes können Container automatisch skaliert und verwaltet werden, um den Anforderungen der Anwendung gerecht zu werden.

## 5.2 Microservices

Die Microservice-Architektur ist ein Ansatz zur Softwareentwicklung, bei dem eine Applikation als Sammlung unabhängiger Komponenten aufgebaut wird, wobei jede als eigenständiger Service läuft. Diese Services kommunizieren über definierte Schnittstellen mittels APIs. Jeder Service ist auf eine spezifische Aufgabe ausgerichtet und kann unabhängig von den anderen aktualisiert, deployt und skaliert werden.[5]

### 5.2.1 Vorteile von Microservices

- **Flexible Skalierung:** Jeder Service kann unabhängig von den anderen skaliert werden, um der Nachfrage gerecht zu werden, ohne das gesamte System zu skalieren.
- **Technologische Freiheit:** Teams können für jeden Service die passendste Technologie wählen, unabhängig von den Technologie-Stacks anderer Services. Zum Beispiel könnte ein neuer Service für diese DA in Rust implementiert werden, während sich die anderen Services nicht ändern müssen.
- **Einfaches Deployment:** Änderungen an einem Service können unabhängig deployt werden, ohne das gesamte System neu zu deployen. Das ermöglicht schnellere Iterationen und häufigere Updates.
- **Resilienz:** Falls ein einzelner Service ausfällt, kann die Anwendung weiterhin funktionieren, wenn auch mit eingeschränkter Funktionalität.[5]

### 5.2.2 Nutzung von Microservices in dieser DA

In dieser DA wurde der Backend-API-Server als Microservice-Architektur umgesetzt. Er ist in mehrere unabhängige Services unterteilt, die jeweils über eine eigene Datenbankinstanz verfügen und über REST-APIs miteinander kommunizieren. Alle Anfragen der Webanwendung werden über ein zentrales Nginx-API-Gateway geleitet, welches die Anfragen an den jeweils zuständigen Service weiterleitet.

## 6 Technologieauswahl

### 6.1 GitHub

Als Versionskontrollplattform wurde GitHub gewählt, welches auf dem verteilten Versionskontrollsystem Git basiert und kollaborative Softwareentwicklung durch Branching, Pull Requests und Code-Reviews ermöglicht.[6] Im Vergleich zu Alternativen wie GitLab oder Bitbucket bietet GitHub die größte Entwickler-Community und eine nahtlose Integration mit zahlreichen Entwicklungstools.

### 6.2 Nginx

Als API-Gateway wurde Nginx gewählt, welches eingehende HTTP-Anfragen entgegennimmt, die JSON Web Token (JWT)-Authentifizierung über das `auth_request`-Modul übernimmt und Anfragen an den jeweils zuständigen Microservice weiterleitet.[7] Im Vergleich zu Alternativen wie Traefik oder HAProxy bietet Nginx eine besonders hohe Stabilität sowie eine flexible Konfigurierbarkeit. Gegenüber einem vollwertigen API-Gateway wie Kong bringt Nginx deutlich weniger Overhead mit und ist damit besser für kleine Microservice-Deployments geeignet. In der DA übernimmt Nginx die zentrale Rolle als einziger Einstiegspunkt für alle Anfragen der Webanwendung und stellt sicher, dass kein Service ohne gültige Authentifizierung erreichbar ist.

### 6.3 Docker

Zur Containerisierung wurde Docker gewählt, welches, wie in Kapitel 5.1 detailliert beschrieben, die plattformunabhängige Bereitstellung aller Systemkomponenten ermöglicht. [2] Im Vergleich zu alternativen Container-Runtimes wie Podman ist Docker durch seine weite Verbreitung und die umfangreiche Dokumentation besonders gut geeignet. Die Containerisierung ermöglicht das Hosting aller Services auf unterschiedlichen Plattformen, von einem Raspberry Pi bis hin zu einem Server, ohne dass Änderungen am Code notwendig sind.

## 6.4 PostgreSQL

Für die Datenhaltung wurde PostgreSQL als relationale Datenbank gewählt, welche ACID-Konformität, komplexe Abfragen und strukturierte Datenbeziehungen unterstützt.[8] Während NoSQL-Datenbanken wie MongoDB für dokumentenbasierte oder hochskalierbare Anwendungen Vorteile bieten, fällt die Entscheidung aufgrund der strukturierten Beziehungen zwischen den Daten im vorliegenden Fall auf eine relationale Datenbank. Gegenüber MySQL bietet PostgreSQL eine strengere Standardkonformität sowie bessere Unterstützung für komplexe Datentypen und Constraints, was die Datenintegrität über alle Microservices hinweg sicherstellt.

## 6.5 Keycloak

Für die Authentifizierung und Benutzerverwaltung wurde Keycloak als dedizierter Identity Provider gewählt, welcher OAuth2- und OpenID-Connect-basierte Authentifizierung sowie rollenbasierte Zugriffskontrolle (RBAC) out-of-the-box bereitstellt.[9] Im Vergleich zur Eigenimplementierung reduziert Keycloak den Entwicklungsaufwand erheblich und bietet gleichzeitig ein höheres Sicherheitsniveau durch standardkonforme Protokolle. Gegenüber einfacheren Lösungen wie Auth0 bietet Keycloak den Vorteil, dass es vollständig mit Docker selbst gehostet werden kann. In der DA übernimmt Keycloak die Ausstellung und Validierung von JWTs.

## 6.6 SQLAlchemy

Als Object-Relational-Mapper (ORM) wurde SQLAlchemy gewählt, welches den Zugriff auf die PostgreSQL-Datenbank über Python-Objekte abstrahiert und direkte Structured Query Language (SQL)-Abfragen überflüssig macht.[10] Im Vergleich zu reinen SQL-Ansätzen reduziert SQLAlchemy Boilerplate-Code erheblich, verbessert die Lesbarkeit und ermöglicht durch das deklarative Modellsystem eine klare Abbildung der Datenbankstruktur auf Python-Klassen.

## 6.7 FastAPI

Als Web-Framework wurde FastAPI gewählt. FastAPI ist ein modernes Python-Web-Framework, welches automatisch API-Dokumentation generiert.[11] Im Vergleich zu anderen Python-API-Frameworks wie Flask bietet FastAPI native Type-Hints-Unterstützung und automatische Validierung von Request- und Response-Daten durch Pydantic.[12] Verglichen mit Django bringt FastAPI deutlich weniger Overhead mit und ist damit besser für kleine Micro-services geeignet.[12] In der DA bildet FastAPI die Grundlage jedes einzelnen Backend-Services und stellt die jeweiligen REST-Endpunkte zur Verfügung, über die die Webanwendung und die Services untereinander kommunizieren.

## 6.8 Next.js

Als Frontend-Framework wurde Next.js gewählt, welches auf React basiert und sowohl serverseitiges Rendering (SSR) als auch statische Seitengenerierung (SSG) unterstützt.[13] Im Vergleich zu anderen Frameworks spricht für Next.js die bessere Ladezeit sowie die vereinfachte Projektstruktur durch das dateibasierte Routing-System. In der DA wird Next.js für die Webanwendung eingesetzt, über die die Nutzer auf das System zugreifen.

## 6.9 shadcn/ui

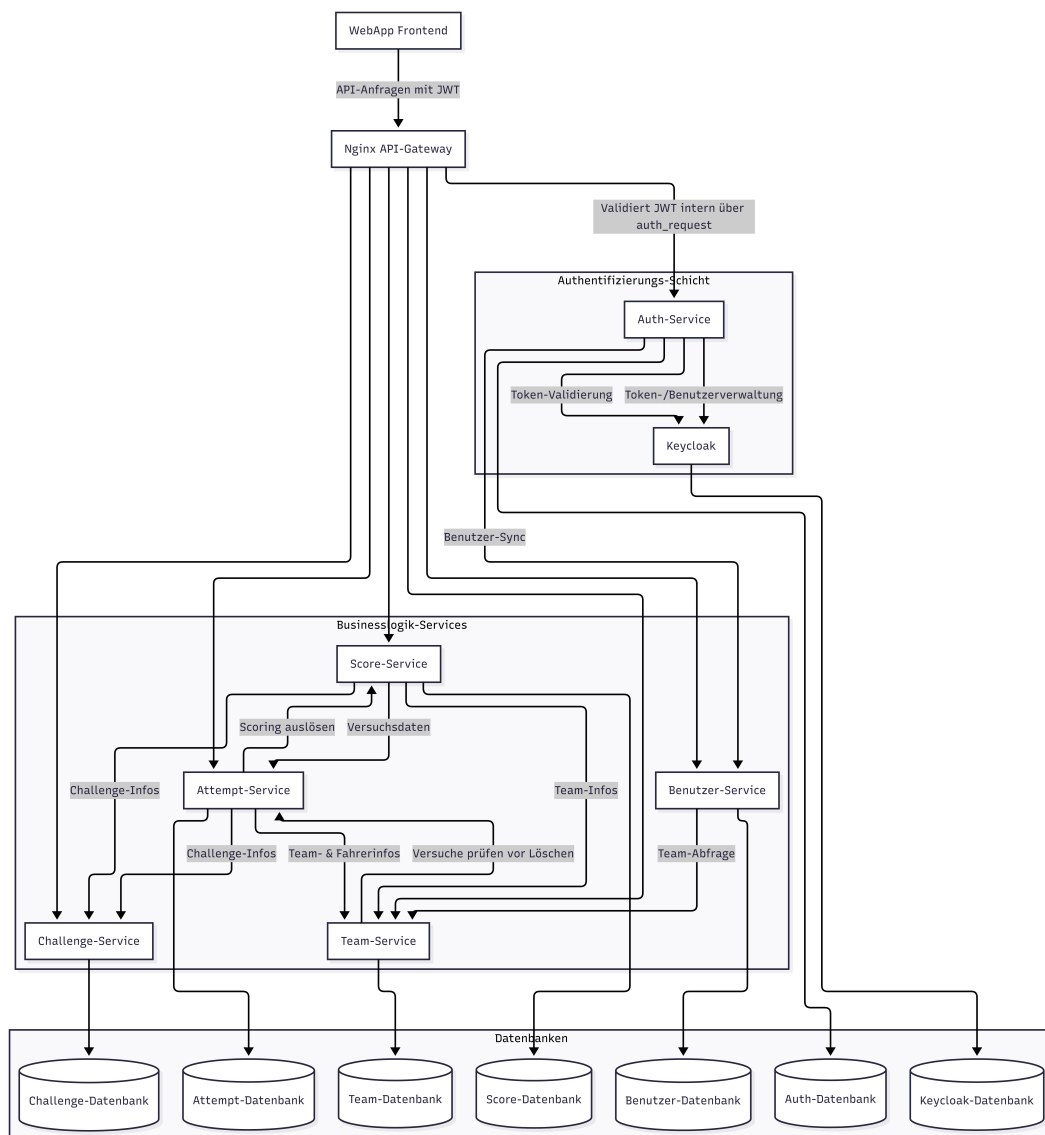
Als UI-Bibliothek wurde shadcn/ui gewählt, welches auf Radix UI und Tailwind CSS basiert und vorgefertigte, vollständig anpassbare Komponenten bereitstellt.[14] Gegenüber einer Selbstimplementierung reduziert shadcn/ui den Programmieraufwand für häufig benötigte UI-Elemente wie Formulare, Dialoge und Tabellen erheblich, ohne dabei die Flexibilität beim Styling einzuschränken.



## 7 Systemarchitektur

### 7.1 Überblick

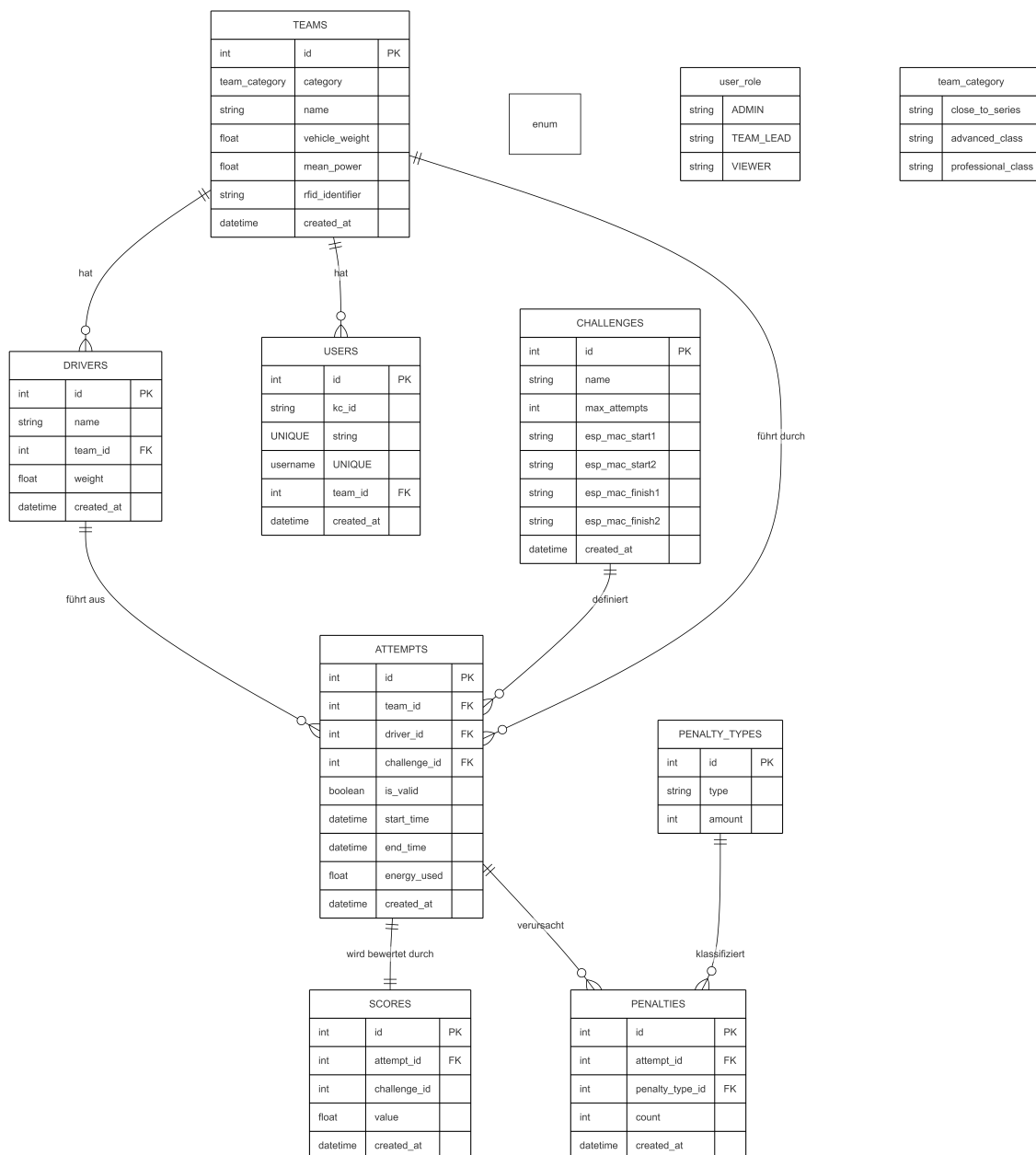
Das Zero Emission Challenge (ZEC)-Timingsystem besteht aus zwei zentralen Teilen: einer webbasierten Anwendung, welche zur Anzeige der Renndaten und zur Verwaltung der Teams und Fahrer dient, sowie einem Backend-Server, welcher als Microservice-Architektur umgesetzt ist. Der Server übernimmt die gesamte Businesslogik, die Authentifizierung, die Datenverwaltung und die Kommunikation mit der Webanwendung und anderen externen Systemen. Abbildung 2 zeigt einen Überblick über alle Komponenten des Servers und deren Zusammenspiel intern sowie nach außen.



**Bild 2:** Systemarchitektur

## 7.2 Server

Die Microservices kommunizieren untereinander über API, wenn sie Daten von anderen Services benötigen. Benötigt z. B. der Attempt Service Informationen über ein Team, fragt er diese beim Team Service ab. Jeder Service verwaltet seine Daten in seiner eigenen PostgreSQL-Datenbank. Da jeder Service eine eigene Datenbank besitzt, existieren die im Gesamtschema 3 dargestellten Beziehungen zwischen den Tabellen nicht als echte Fremdschlüssel, sondern werden in den Services sichergestellt.



**Bild 3:** Datenbankschema des ZEC-Timing-Systems

### 7.2.1 Nginx API-Gateway

Das API-Gateway bildet den einzigen Zugang zum System und ist für das Routing, die Authentifizierung sowie die CORS-Konfiguration aller Anfragen zuständig.

#### Routing

Das Routing erfolgt pfadbasiert; jeder Pfad wird einem bestimmten Service zugeordnet:

- `/users` → User Service
- `/teams`, `/drivers` → Team Service
- `/challenges` → Challenge Service
- `/attempts` → Attempt Service
- `/scores`, `/leaderboard`, `/penalties` → Score Service
- `/login`, `/refresh` → Auth Service

#### Authentifizierung

Bei jeder weitergeleiteten Anfrage wird ein Subrequest an den Auth Service gesendet, welcher den angehängten JWT validiert und die Rolle des Users prüft. Schlägt die Validierung fehl, wird die Anfrage abgewiesen, ohne die anderen Services zu treffen. Anfragen mit einem gültigen API-Key bilden eine Ausnahme, sie sind für die Kommunikation mit anderen Applikationen im System vorgesehen.

#### Header-Weitergabe

Nach erfolgreicher Authentifizierung fügt Nginx die Userdaten als HTTP-Header der weitergeleiteten Anfrage hinzu. Services können so auf die Identität des Nutzers zugreifen, ohne Tokenvalidierung durchführen zu müssen.

#### CORS

Für alle Endpunkte werden die CORS-Header gesetzt, sodass die Webanwendung vom Browser aus auf die API zugreifen kann. Preflight-Anfragen werden direkt von Nginx beantwortet, ohne dass ein Backend-Service involviert wird.

### 7.2.2 Auth Service

Der Auth Service übernimmt jegliche Authentifizierung im System und ist eng mit Keycloak als Identity Provider verbunden.

#### Interne Verifikation

Der Auth Service stellt drei interne Endpunkte bereit, welche ausschließlich vom API-Gateway aufgerufen werden. Es gibt pro Nutzerrolle (Administrator, Teamleiter, Viewer) einen Endpunkt, der überprüft, ob der Nutzer die entsprechende Rolle besitzt.

#### Login & Token-Refresh

Öffentlich stellt der Auth Service zwei Endpunkte bereit: einen Login-Endpunkt und einen Refresh-Endpunkt. Diese dienen der Authentifizierung sowie der Erneuerung eines abgelaufenen Tokens.

### 7.2.3 User Service

Der User Service verwaltet die Nutzerkonten außerhalb von Keycloak und agiert als Bindeglied zwischen Keycloak und der restlichen Anwendung. Er speichert nur systeminterne Daten, während Passwörter und Rollen in Keycloak liegen.

#### Datenbankschema

| Spalte     | Typ             | Beschreibung                |
|------------|-----------------|-----------------------------|
| id         | Integer (PK)    | Interne ID des Benutzers    |
| kc_id      | String (UNIQUE) | Keycloak-ID des Benutzers   |
| username   | String (UNIQUE) | Benutzername                |
| team_id    | Integer         | Zugehöriges Team (optional) |
| created_at | DateTime        | Erstellungszeitpunkt        |

**Tabelle 1:** Datenbankschema: User Service

Die `kc_id` stellt die Verbindung zwischen dem lokalen Datenbankdatensatz und dem entsprechenden Keycloak-Benutzerkonto her. Wie in 7.2 beschrieben, ist `team_id` kein echter Fremdschlüssel, sondern wird in der Businesslogik des User Service sichergestellt.

#### Funktionalität

Der User Service ermöglicht das Erstellen, Aktualisieren und Löschen von Benutzern, wobei Änderungen stets sowohl in Keycloak als auch in der lokalen Datenbank durchgeführt werden. Darüber hinaus können Benutzern Rollen in Keycloak zugewiesen oder entzogen werden. Für Operationen in Keycloak bezieht der User Service über den Auth Service einen kurzlebigen Admin-Token.

### 7.2.4 Team Service

Der Team Service verwaltet die teilnehmenden Teams sowie deren Fahrer und stellt diese Daten den anderen Services über seine REST-API zur Verfügung.

#### Datenbankschema

| Spalte            | Typ          | Beschreibung                    |
|-------------------|--------------|---------------------------------|
| id                | Integer (PK) | Interne ID des Teams            |
| name              | String       | Name des Teams                  |
| category          | Enum         | Teamkategorie                   |
| vehicle_weight    | Float        | Fahrzeuggewicht in kg           |
| mean_power        | Float        | Durchschnittliche Leistung in W |
| rfid_identifizier | String       | RFID-Kennung des Fahrzeugs      |
| created_at        | DateTime     | Erstellungszeitpunkt            |

**Tabelle 2:** Datenbankschema: Teams

| Spalte     | Typ          | Beschreibung              |
|------------|--------------|---------------------------|
| id         | Integer (PK) | Interne ID des Fahrers    |
| name       | String       | Name des Fahrers          |
| team_id    | Integer      | Zugehöriges Team          |
| weight     | Float        | Gewicht des Fahrers in kg |
| created_at | DateTime     | Erstellungszeitpunkt      |

**Tabelle 3:** Datenbankschema: Drivers

#### Funktionalität

Der Team Service bietet die Funktionalität zum Erstellen, Lesen, Aktualisieren und Löschen von Teams und Fahrern. Greift ein User mit der Rolle `TEAM_LEAD` auf die API zu, wird überprüft, ob er auf sein eigenes Team zugreift, andernfalls wird die Anfrage abgelehnt.

### 7.2.5 Challenge Service

Der Challenge Service verwaltet die einzelnen Bewerbe der ZEC und stellt deren Konfiguration den anderen Services zur Verfügung. Ein Bewerb definiert die maximale Anzahl an Versuchen sowie die MAC-Adressen der zugehörigen Lichtschranken.

#### Datenbankschema

| Spalte          | Typ          | Beschreibung                     |
|-----------------|--------------|----------------------------------|
| id              | Integer (PK) | Interne ID des Bewerbs           |
| name            | String       | Name des Bewerbs                 |
| max_attempts    | Integer      | Maximale Anzahl an Versuchen     |
| esp_mac_start1  | String       | MAC-Adresse Startlichtschranke 1 |
| esp_mac_start2  | String       | MAC-Adresse Startlichtschranke 2 |
| esp_mac_finish1 | String       | MAC-Adresse Ziellichtschranke 1  |
| esp_mac_finish2 | String       | MAC-Adresse Ziellichtschranke 2  |
| created_at      | DateTime     | Erstellungszeitpunkt             |

**Tabelle 4:** Datenbankschema: Challenge Service

#### Funktionalität

Der Challenge Service bietet nur Leseoperationen und Aktualisierungen an, da die Bewerbe einmalig per Datenbankskript angelegt und danach nur noch aktualisiert werden. GET-Anfragen sind öffentlich zugänglich, sodass die Ergebnisanzeige ohne Anmeldung auf Bewerbungsinformationen zugreifen kann. Schreibende Operationen sind ausschließlich Administratoren erlaubt.

### 7.2.6 Attempt Service

Der Attempt Service ist für die Erfassung und Verwaltung der Versuche zuständig. Er hat eine Vielzahl von Inter-Service-Abhängigkeiten, siehe 2, da viele Daten verifiziert werden müssen. Außerdem werden automatisch die Punkteberechnung sowie die Strafpunkte im Score Service ausgelöst.

#### Datenbankschema

| Spalte       | Typ          | Beschreibung              |
|--------------|--------------|---------------------------|
| id           | Integer (PK) | Interne ID des Versuchs   |
| team_id      | Integer      | Zugehöriges Team          |
| driver_id    | Integer      | Zugehöriger Fahrer        |
| challenge_id | Integer      | Zugehöriger Bewerb        |
| is_valid     | Boolean      | Gültigkeit des Versuchs   |
| start_time   | DateTime     | Startzeitpunkt            |
| end_time     | DateTime     | Endzeitpunkt              |
| energy_used  | Float        | Verbrauchte Energie in Wh |
| created_at   | DateTime     | Erstellungszeitpunkt      |

**Tabelle 5:** Datenbankschema: Attempt Service

#### Funktionalität

Beim Erstellen eines Versuchs werden Team, Fahrer und Bewerb validiert sowie überprüft, ob das Team die maximale Versuchsanzahl bereits erreicht hat. Nach dem Speichern werden optional Strafpunkte sowie automatisch ein Score-Eintrag im Score Service angelegt. Wird ein Versuch nachträglich als ungültig markiert, wird der zugehörige Score automatisch entfernt. Beim Löschen eines Versuchs werden Score und Strafpunkte ebenfalls gelöscht.

#### Export

Über einen Export-Endpoint können alle Versuche eines Bewerbs als CSV- oder XLSX-Datei heruntergeladen werden. Die Daten werden dabei mit Team- und Fahrerinformationen verbunden und können optional nach Teamkategorie gefiltert werden.

### 7.2.7 Score Service

Der Score Service ist für die Berechnung und Verwaltung der Punktestände sowie die Bereitstellung der Ergebnislisten zuständig. Er verwaltet neben den Scores auch die Strafpunkte und implementiert die bewerbungsspezifische Punkteberechnungslogik.

#### Datenbankschema

| Spalte       | Typ          | Beschreibung          |
|--------------|--------------|-----------------------|
| id           | Integer (PK) | Interne ID des Scores |
| attempt_id   | Integer      | Zugehöriger Versuch   |
| challenge_id | Integer      | Zugehöriger Bewerb    |
| value        | Float        | Berechneter Punktwert |
| created_at   | DateTime     | Erstellungszeitpunkt  |

**Tabelle 6:** Datenbankschema: Scores

| Spalte          | Typ          | Beschreibung          |
|-----------------|--------------|-----------------------|
| id              | Integer (PK) | Interne ID der Strafe |
| attempt_id      | Integer      | Zugehöriger Versuch   |
| penalty_type_id | Integer      | Art der Strafe        |
| count           | Integer      | Anzahl der Strafen    |
| created_at      | DateTime     | Erstellungszeitpunkt  |

**Tabelle 7:** Datenbankschema: Penalties

| Spalte | Typ          | Beschreibung             |
|--------|--------------|--------------------------|
| id     | Integer (PK) | Interne ID des Straftyps |
| type   | String       | Bezeichnung der Strafe   |
| amount | Integer      | Strafzeit in Sekunden    |

**Tabelle 8:** Datenbankschema: Penalty Types

#### Punkteberechnung

Die Punkteberechnung ist pro Bewerb unterschiedlich. Die Berechnungsformeln der einzelnen Bewerbe werden im Kapitel zur Score-Implementierung im Detail beschrieben.

#### Leaderboard

Pro Bewerb und Teamkategorie gibt es eine Ergebnisliste, in der jedes Team mit seinem besten Score angeführt wird. Diese Listen können als CSV- oder XLSX-Datei exportiert werden. GET-Anfragen auf die Leaderboard-Endpunkte sind öffentlich zugänglich, sodass die Ergebnisanzeige ohne Anmeldung auf die Daten zugreifen kann.



## 7.3 Web App

Die Webanwendung ist mit Next.js[13] umgesetzt und bildet die Benutzeroberfläche des ZEC-Timing-Systems. Sie kommuniziert über das Nginx API-Gateway mit dem Backend und passt ihre Anzeige dynamisch an die Rolle des angemeldeten Benutzers an.

### Authentifizierung & Sitzungsverwaltung

Die Anmeldung erfolgt über den Login-Tab, auf dem Benutzername und Passwort eingegeben werden. Nach erfolgreicher Anmeldung werden Access- und Refresh-Token lokal gespeichert und bei jeder API-Anfrage mitgesendet. Läuft die Sitzung ab, wird der Benutzer automatisch zur Login-Seite weitergeleitet und über das Sitzungsende informiert.

### Rollenbasierte Ansichten

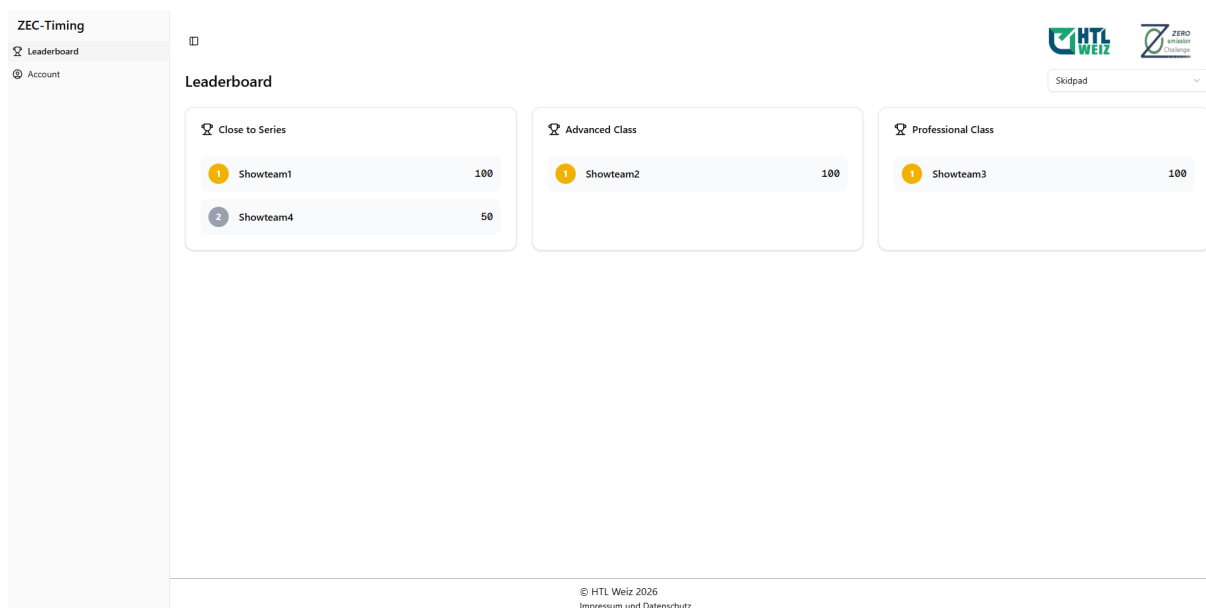
Die Webanwendung unterscheidet drei Benutzerrollen mit unterschiedlichen Tabs:

- **Viewer:** Hat nur Zugriff auf die öffentliche Ergebnisliste (*Leaderboard*), die die aktuellen Punktestände aller Teams pro Bewerb und Teamkategorie anzeigt. Dieser Bereich ist auch ohne Anmeldung zugänglich.
- **Teamleiter:** Sieht nach der Anmeldung eine personalisierte Teamansicht mit den Daten des eigenen Teams und eine Übersicht der eigenen Fahrer. Teamleiter können Teamdaten und Fahrer bearbeiten, jedoch ausschließlich innerhalb ihres eigenen Teams.
- **Administrator:** Hat Zugriff auf alle Tabs der Anwendung. Dazu zählen die Verwaltung von Benutzern und Rollen, die Verwaltung aller Teams und Fahrer, die Einsicht und Bearbeitung aller Versuche, die Konfiguration der Bewerbe sowie der Export von Ergebnissen und Versuchen.

## Tabs

Die Anwendung ist in folgende Tabs unterteilt, die je nach Rolle sichtbar sind:

- **Leaderboard:** Zeigt die Ergebnislisten aller Bewerbe, gefiltert nach Teamkategorie. Für jeden Bewerb wird ein eigenes Ranking mit Punktestand angezeigt.
- **Attempts:** Ermöglicht Administratoren die Einsicht, Bearbeitung und Löschung aller Zeitversuche. Start- und Endzeit sowie Energieverbrauch können nachträglich korrigiert und Versuche als ungültig markiert werden.
- **Teams:** Bietet Administratoren eine vollständige Übersicht aller Teams inklusive Fahrerliste. Teamleiter sehen hingegen ausschließlich ihr eigenes Team in einer detaillierten Einzelansicht.
- **Challenges:** Erlaubt Administratoren die Konfiguration der Bewerbe.
- **Users:** Ermöglicht Administratoren das Erstellen, Bearbeiten und Löschen von Benutzern sowie die Verwaltung von Rollenzuweisungen.
- **Export:** Stellt Administratoren den Export der Ergebnislisten und Zeitversuche als CSV- oder XLSX-Datei zur Verfügung, gefiltert nach Bewerb und optional nach Teamkategorie.



**Bild 4:** Leaderboard-Tab für Viewer

ZEC-Timing

Leaderboard

Teams

Account

Showteam1

Team Details

Category

Close to Series

Vehicle Weight

10 kg

Mean Power

10 W

RFID Identifier

123456789

Edit Team

Team Drivers

+ Add Driver

Simon

Weight: 10 kg

HTL WEIZ

ZERO Emission Challenge

© HTL Weiz 2026

Impressum und Datenschutz

Bild 5: Teams-Tab für Teamleiter

ZEC-Timing

Leaderboard

Attempts

Teams

Challenges

Users

Export

Account

Teams & Drivers Management

All Teams

| Team Name | Category        | Weight (kg) | Power (W) | RFID      | Actions                                      |
|-----------|-----------------|-------------|-----------|-----------|----------------------------------------------|
| Showteam1 | Close to Series | 10          | 10        | 123456789 | <div><div></div><div></div><div></div></div> |

Drivers

Simon

Weight: 10kg

HTL WEIZ

ZERO Emission Challenge

© HTL Weiz 2026

Impressum und Datenschutz

Bild 6: Teams-Tab für Administrator

2025/26- 5AHIT- David Darnhofer

27 | 51

## 8 Implementation

### 8.1 Server

#### 8.1.1 Zugriffsberechtigungen

Die Zugriffskontrolle im Backend basiert auf FastAPIs Dependency-Injection-System. Rollen werden einmal definiert und dann über die gesamte Anwendung verwendet. Die Hierarchie ist so aufgebaut, dass höhere Rollen automatisch die Berechtigungen der darunterliegenden Rollen einschließen.

```
1  ROLE_HIERARCHY = {
2      UserRole.VIEWER: {
3          UserRole.VIEWER,
4          UserRole.TEAM_LEAD,
5          UserRole.ADMIN,
6      },
7      UserRole.TEAM_LEAD: {
8          UserRole.TEAM_LEAD,
9          UserRole.ADMIN,
10     },
11     UserRole.ADMIN: {
12         UserRole.ADMIN,
13     },
14 }
15
16 def require_role(required_role: UserRole):
17     def role_checker(user: CurrentUser):
18         user_roles = {UserRole(role) for role in user["roles"]}
19         allowed_roles = ROLE_HIERARCHY[required_role]
20         if user_roles.isdisjoint(allowed_roles):
21             raise exception.InsufficientPermissions
22         return user
23     return role_checker
24
25 AdminUser = Annotated[dict, Depends(require_role(UserRole.ADMIN))]
26 TeamLeadUser = Annotated[dict, Depends(require_role(UserRole.TEAM_LEAD))]
27 ViewerUser = Annotated[dict, Depends(require_role(UserRole.VIEWER))]
```

Die oben definierten Typen können direkt als Parameter in Endpunkt-Funktionen verwendet werden. FastAPI löst die Dependency automatisch auf und führt die Rollenprüfung vor der eigentlichen Endpunktlogik aus.

```
1 @router.get("/internal/verify/teamlead")
2 def verify_teamlead(request: Request, response: Response,
3                     current_user: TeamLeadUser):
4     user_resp = requests.get(
5         f"{USER_URL}/api/users/me",
6         headers={"Authorization": request.headers.get("Authorization")}
7     )
8     user_data = user_resp.json()
9     response.headers["X-User-Id"] = current_user["sub"]
10    response.headers["X-Team-Id"] = str(user_data.get("team_id"))
11    response.headers["X-Role"] = current_user["roles"][0]
12    return {"active": True, **current_user}
```

Der `verify_teamlead`-Endpunkt hat eine Besonderheit. Er sendet die Team-ID des Benutzers als HTTP-Antwort-Header zurück, damit Nginx diese Informationen an die Ziel-Services weiterreichen kann.

### 8.1.2 API-Gateway

Das Nginx API-Gateway wird über eine zentrale Konfigurationsdatei definiert. Die grundlegende Struktur besteht aus der Definition der Upstream-Services sowie einem Server-Block, der alle Routing- und Authentifizierungsregeln enthält.

#### Upstream-Konfiguration

Jeder Microservice wird als eigener Upstream registriert, damit Nginx die Anfragen richtig weiterleiten kann. Hier der User Service als Beispiel:

```
1 upstream user_service {server user-service:8001;}
```

## Interne Authentifizierungsendpunkte

Die Authentifizierung wird über drei interne Endpunkte realisiert, die ausschließlich vom auth\_request-Modul aufgerufen werden und von außen nicht erreichbar sind. Der einzige Unterschied zwischen den drei ist die URL, die auf den Auth Service zeigt, welcher die jeweilige Rolle überprüft. Hier der Endpunkt für die Admin-Authentifizierung als Beispiel:

```
1 location = /auth_admin {
2     internal;
3     proxy_pass http://auth_service/api/auth/internal/verify/admin;
4     proxy_pass_request_body off;
5     proxy_set_header Content-Length "";
6 }
```

## Routing mit Authentifizierung

Jeder Endpunkt verwendet auth\_request, um die Berechtigung zu prüfen. Das folgende Beispiel zeigt das Routing für den User Service, der ausschließlich Administratoren zugänglich ist:

```
1 location ~ ^/users(/.*)?$ {
2     auth_request /auth_admin;
3     proxy_pass http://user_service/api/users$1$is_args$args;
4 }
```

## API-Key Bypass

Für die geplante Anbindung an die Zeitnehmer-App kann die Authentifizierung mittels API-Key umgangen werden:

```
1 map $http_x_api_key $skip_auth {
2     default 0;
3     "this-is-not-the-key" 1;
4 }
5 location = /auth_admin {
6     internal;
7     if ($skip_auth = 1) { return 204; }
8     proxy_pass http://auth_service/api/auth/internal/verify/admin;
9 }
```

### 8.1.3 CRUD-Operationen

Die Create, Read, Update, Delete (CRUD)-Operationen sind in allen Services nach demselben Muster aufgebaut: Ein Router-Modul definiert die HTTP-Endpunkte und delegiert die eigentliche Logik an ein separates CRUD-Modul. Dieses Muster wird am Beispiel des Team Service erläutert.

#### Router

Der Router definiert die Endpunkte und deren HTTP-Methoden. Die Datenbankverbindung wird über FastAPIs Dependency-Injection als `SessionDep` bereitgestellt, sie wird automatisch geöffnet, an den Endpunkt übergeben und nach dem Request wieder geschlossen. Auf der HTTP-Ebene ändert sich dann immer nur die Methode, die der Router aufruft (POST, GET, PUT, DELETE). Hier der Create-Endpunkt des Team Service als Beispiel:

```
1 @router.post("/", response_model=TeamResponse)
2 def create_team(db: SessionDep, team: TeamCreate):
3     return crud.create_team(db=db, team=team)
```

#### Erstellen

Beim Erstellen eines Teams wird das eingehende Pydantic-Schema direkt in ein SQLAlchemy-Modell umgewandelt und in der Datenbank gespeichert. Bei einem Fehler wird die Transaktion zurückgerollt:

```
1 def create_team(*, db: SessionDep, team: TeamCreate):
2     try:
3         db_team = Team(**team.model_dump(exclude_unset=True))
4         db.add(db_team)
5         db.commit()
6         db.refresh(db_team)
7         return db_team
8     except Exception as exc:
9         db.rollback()
10    raise ServiceError() from exc
```

## Aktualisieren

Beim Aktualisieren werden nur die Felder überschrieben, die tatsächlich in der Anfrage enthalten sind. Das `exclude_unset=True`-Flag von Pydantic stellt sicher, dass nicht gesetzte Felder nicht auf ihre Standardwerte zurückgesetzt werden:

```
1 def update_team(*, db: SessionDep, team_id: int,
2                 team_update: TeamUpdate, request: Request):
3     check_team_permissions(db=db, team_id=team_id, request=request)
4     try:
5         db_team = get_team(db=db, team_id=team_id, request=request)
6         update_data = team_update.model_dump(
7             exclude_unset=True,
8             exclude={"id"},
9         )
10        for field, value in update_data.items():
11            setattr(db_team, field, value)
12        db.commit()
13        db.refresh(db_team)
14        return db_team
15    except EntityDoesNotExistError:
16        db.rollback()
17        raise
18    except Exception as exc:
19        db.rollback()
20        raise ServiceError() from exc
```



## Löschen

Beim Löschen wird das angeforderte Team aus der Datenbank entfernt. Zuvor wird eine Berechtigungsprüfung durchgeführt, um sicherzustellen, dass der Zugriff erlaubt ist.

```
1 def delete_team(*, db: SessionDep, team_id: int, request: Request):
2     check_team_permissions(db=db, team_id=team_id, request=request)
3     try:
4         db_team = get_team(db=db, team_id=team_id, request=request)
5         db.delete(db_team)
6         db.commit()
7         return db_team
8     except EntityDoesNotExistError:
9         db.rollback()
10        raise
11    except Exception as exc:
12        db.rollback()
13        raise ServiceError() from exc
```

## Rollenbasierte Berechtigungsprüfung

Zugriffe werden zusätzlich durch eine serviceinterne Berechtigungsprüfung abgesichert. Teamleiter dürfen ausschließlich auf ihr eigenes Team zugreifen, die Team-ID des anfragenden Benutzers wird dabei aus dem vom Nginx API-Gateway weitergeleiteten X-Team-Id-Header ausgelesen:

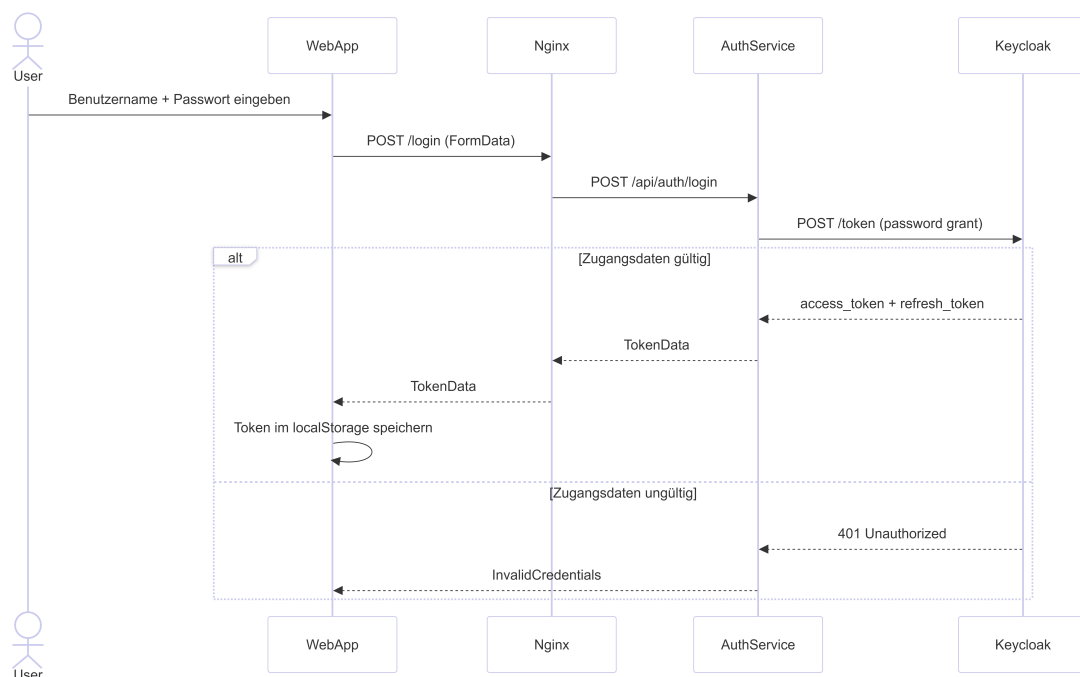
```
1 def check_team_permissions(*, db: SessionDep,
2                             team_id: int | None = None,
3                             request: Request):
4     role = request.headers.get("X-Role")
5     user_team_id = request.headers.get("X-Team-Id")
6     if role == "TEAM_LEAD" and team_id is not None:
7         if int(team_id) != int(user_team_id):
8             raise InsufficientPermissions(
9                 "Teamleads can only operate on their own team."
10            )
```

### 8.1.4 Authentifizierung mit Keycloak

Die Authentifizierung funktioniert mit JWT-Tokens, die von Keycloak ausgestellt und vom Auth Service validiert werden. Keycloak übernimmt dabei die vollständige Verwaltung von Benutzerkonten, Passwörtern und Rollen. Der Auth Service fungiert als Vermittler zwischen Keycloak und dem restlichen System.

#### Login

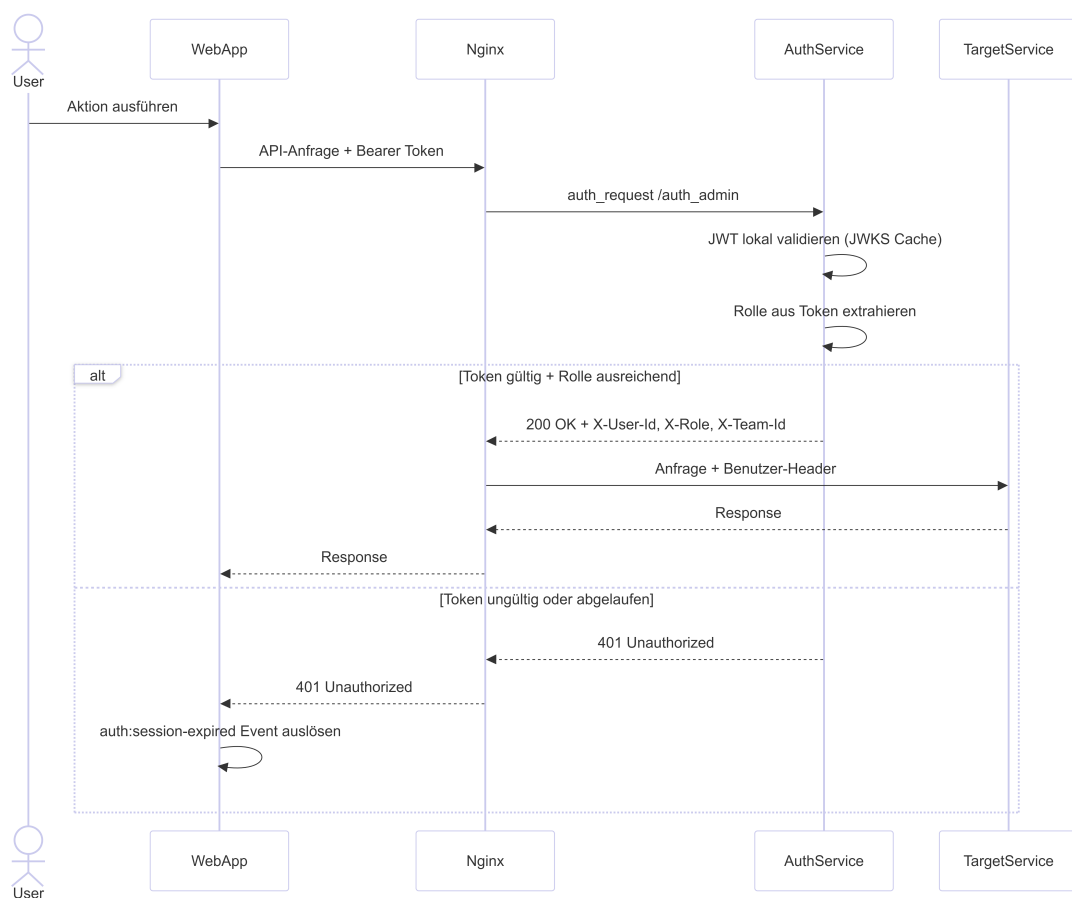
Beim Login sendet die Webanwendung Benutzername und Passwort an den Auth Service, der diese über den OAuth2 Resource Owner Password Flow an Keycloak weiterleitet. Bei Erfolg liefert Keycloak ein Access- sowie ein Refresh-Token zurück, die in der Webanwendung im localStorage gespeichert werden.



**Bild 7:** Sequenzdiagramm: Login-Ablauf

## Authentifizierte Anfrage

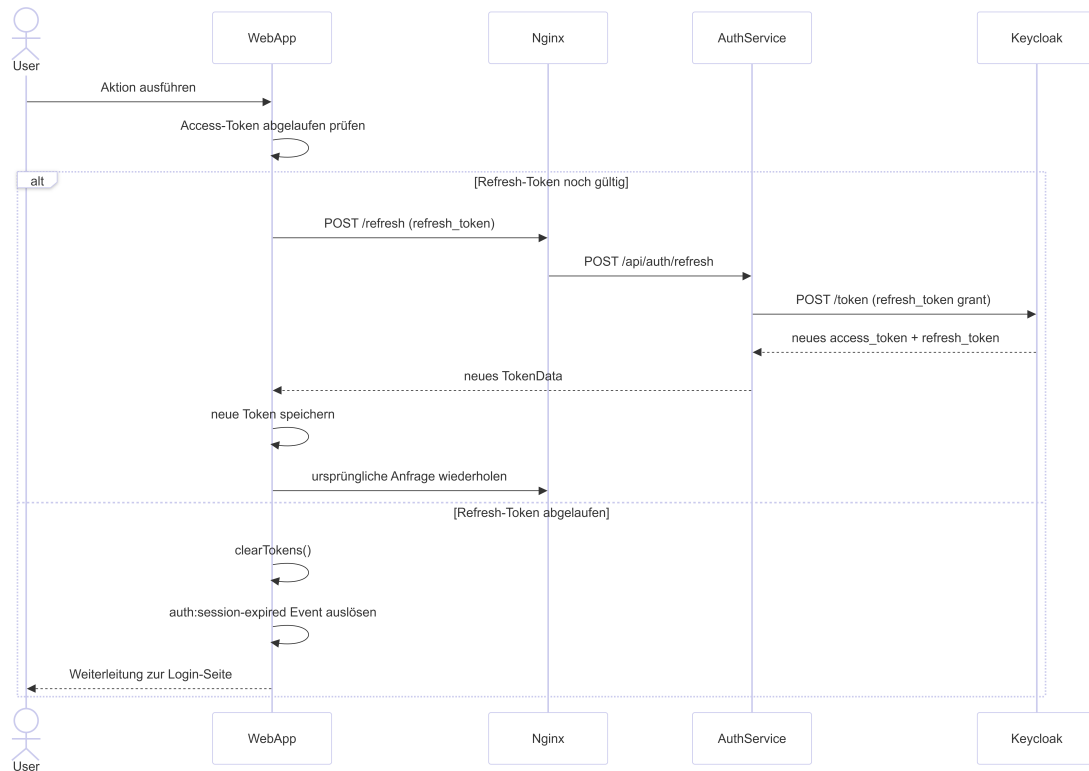
Bei jeder API-Anfrage prüft Nginx über das `auth_request`-Modul, ob der mitgesendete JWT gültig ist. Der Auth Service validiert den Token lokal anhand der gecachten öffentlichen Schlüssel von Keycloak, Keycloak selbst wird dabei nicht eingebunden. Nach erfolgreicher Validierung wird die Anfrage mit den Benutzerinformationen als Header an den Ziel-Service weitergeleitet.



**Bild 8:** Sequenzdiagramm: Authentifizierte Anfrage

## Token-Refresh

Stellt die Webanwendung fest, dass das Access-Token abgelaufen ist, wird vor der eigentlichen Anfrage automatisch ein Token-Refresh durchgeführt. Ist auch das Refresh-Token abgelaufen, wird die Sitzung beendet und der Benutzer zur Login-Seite weitergeleitet.



**Bild 9:** Sequenzdiagramm: Token-Refresh

### 8.1.5 Inter-Service-Kommunikation

Die Services kommunizieren untereinander über synchrone HTTP-Aufrufe mittels der Python-requests-Bibliothek[15]. Die URLs der anderen Services werden dabei über Umgebungsvariablen konfiguriert und beim Start des Containers eingelesen, sodass keine URLs hart im Code hinterlegt sind.

### Validierung von Fremdbeziehungen

Wie in 7.2 beschrieben, existieren keine echten Datenbankfremdschlüssel über Servicegrenzen hinweg. Stattdessen werden Referenzen auf andere Services vor dem Speichern explizit validiert. Der Attempt Service prüft beispielsweise vor dem Erstellen eines Versuchs, ob das referenzierte Team, der Fahrer und der Bewerb tatsächlich existieren:

```
1 def _validate_team(team_id: int):
2     resp = requests.get(f"{TEAM_URL}/api/teams/{team_id}")
3     if resp.status_code == 404:
4         raise EntityDoesNotExistError(f"Team {team_id} does not exist")
5     if resp.status_code in (401, 403):
6         raise AuthenticationFailed(f"Unauthorized to fetch team {team_id}")
7     if resp.status_code != 200:
8         raise ServiceError(f"Failed to fetch team {team_id}: {resp.text}")
9
10 def create_attempt(*, db: SessionDep, attempt: AttemptCreate):
11     _validate_team(attempt.team_id)
12     _validate_driver(attempt.driver_id)
13     _validate_challenge(attempt.challenge_id)
14     # ...
```

### Auslösen von Folgeoperationen

Nach dem Speichern eines Versuchs löst der Attempt Service automatisch die Punkteberechnung im Score Service aus. Werden dabei optional auch Strafpunkte mitgeliefert, werden diese zuerst angelegt:

```
1 if attempt.penalty_count and attempt.penalty_type:
2     penalty_payload = {
3         "attempt_id": db_attempt.id,
4         "count": attempt.penalty_count,
5         "penalty_type_id": attempt.penalty_type,
6     }
7     pen_resp = requests.post(
8         f"{SCORE_URL}/api/penalties/", json=penalty_payload
9
10 score_resp = requests.post(
11     f"{SCORE_URL}/api/scores/",
12     json={"attempt_id": db_attempt.id}
13 )
```

## 8.2 Web App

### 8.2.1 Kommunikation mit dem Server

Die Kommunikation mit dem Backend erfolgt über typisierte API-Client-Module, die für jeden Service ein eigenes Objekt bereitstellen. Alle authentifizierten Anfragen laufen über die zentrale `authenticatedFetch`-Funktion, die den Token automatisch als Authorization-Header beifügt:

```
1 export async function authenticatedFetch(  
2   url: string,  
3   options: RequestInit = {}  
4 ): Promise<Response> {  
5   const token = await AuthService.getValidToken()  
6  
7   if (!token) {  
8     window.dispatchEvent(new Event('auth:session-expired'))  
9     throw new Error('Not authenticated')  
10  }  
11  
12  const headers = new Headers(options.headers)  
13  headers.set('Authorization', `Bearer ${token}`)  
14  if (!(options.body instanceof FormData)) {  
15    headers.set('Content-Type', 'application/json')  
16  }  
17  return fetch(url, { ...options, headers })  
18 }
```

Die API-Client-Module abstrahieren die HTTP-Aufrufe hinter typisierten Funktionen. Das folgende Beispiel zeigt die `createTeam`-Funktion des Team Service:

```
1  async createTeam(data: TeamCreate): Promise<Team> {
2      const response = await authenticatedFetch(`${API_BASE_URL}/teams/`, {
3          method: 'POST',
4          body: JSON.stringify(data),
5      })
6      if (!response.ok) throw new Error('Failed to create team')
7      return response.json()
8  },
```

### 8.2.2 Rollenbasierte Zugriffskontrolle

Die Zugriffskontrolle auf der Clientseite wird über ein zentrales Berechtigungsmodul gesteuert. Für jede Rolle ist definiert, welche Tabs sichtbar und welche Aktionen erlaubt sind. Hier ein Ausschnitt aus der Berechtigungsdefinition:

```
1  const rolePermissions: Record<UserRole, RolePermissions> = {
2      admin: {
3          allowedTabs: ['leaderboard', 'attempts', 'teams',
4                      'challenges', 'users', 'export', 'login'],
5          canViewAttempts: true,
6          canEditTeams: true,
7          canEditUsers: true,
8          canEditChallenges: true,
9          canExport: true,
10     },
```

In der Hauptkomponente wird bei jedem Tab-Wechsel geprüft, ob die Rolle des aktuellen Benutzers den Zugriff erlaubt.

```
1  const handleTabChange = (tab: Tabs) => {
2      if (canAccessTab(currentUser?.role || null, tab)) {
3          setActiveTab(tab)
4      }
5  }
```

## 9 Testen

### 9.1 Teststrategie

Die Teststrategie des Projekts verwendet Unit- und Integrationstests für die Backend-Services sowie End-to-End-Tests für die Webanwendung. Als Testframework für die Backend-Tests wird `pytest`[16] eingesetzt, während für die End-to-End-Tests `Playwright`[17] zum Einsatz kommt. Zur Sicherstellung der als Ziel gesetzten Testabdeckung von über 80% wird `pytest-cov`[18] verwendet. Die Testabdeckung wurde in allen Backend-Services erreicht, im Durchschnitt mit 85%. Um diese zu erreichen, wurden nicht nur die "Happy PathSzenarien, sondern auch die Fehlerfälle getestet.

### 9.2 Pytest

Wie in 9.1 beschrieben, wurden die Backend-Tests mit `pytest` umgesetzt. Für die jeweiligen Tests wird eine SQLite-Testdatenbank[19] verwendet, um eine isolierte Testumgebung zu schaffen. Die Testdaten werden über `pytest-Fixtures`[20] bereitgestellt. Abhängigkeiten von anderen Services werden mit `unittest.mock`[21] bzw. `monkeypatch`[22] simuliert. So kann die Businesslogik der Services isoliert getestet werden.

#### 9.2.1 Unit-Tests

Unit-Tests überprüfen die Funktionalität einzelner Funktionen oder Methoden innerhalb der Services. Sie stellen sicher, dass die Kernlogik erwartungsgemäß funktioniert.

Ein Beispieltest für die Funktion `get_fastest_attempt`:

```
1 def test_fastest_attempt(db, seeded_attempts):
2     attempt = crud.get_fastest_attempt(db=db, challenge_id=1)
3     time = attempt.end_time - attempt.start_time
4     assert time == timedelta(seconds=9)
```



### 9.2.2 Integrationstests

Integrationstests prüfen die HTTP-Schnittstellen des Services mit dem FastAPI `TestClient`[23]. Diese Tests prüfen das Verhalten der API aus Sicht eines externen Clients, indem sie HTTP-Anfragen an die Endpunkte senden und die Antworten validieren.

Ein Beispieltest für das Abrufen aller Attempts:

```
1 def test_list_attempts(client):
2     response = client.get("/api/attempts/")
3     assert response.status_code == 200
4     assert len(response.json()) == 2
```

### 9.3 Playwright

Wie in 9.1 beschrieben, werden die End-to-End-Tests mit Playwright[17] umgesetzt. Ziel dieser Tests ist die Validierung der Webanwendung aus Sicht eines realen Benutzers im Browser. Im Unterschied zu den Backend-Tests, welche die Funktionalität isoliert testen, wird hier das Gesamtsystem getestet.

Ein Beispieltest dafür, ob ein Admin-Benutzer alle Tabs der Webanwendung sehen kann:

```
1 test('admin can log in and see all tabs', async ({ page }) => {
2     await loginAs(page, 'admin', 'changeme');
3
4     await expect(page.getByRole('button', { name: 'Leaderboard' })).toBeVisible();
5     await expect(page.getByRole('button', { name: 'Attempts' })).toBeVisible();
6     await expect(page.getByRole('button', { name: 'Teams' })).toBeVisible();
7     await expect(page.getByRole('button', { name: 'Challenges' })).toBeVisible();
8     await expect(page.getByRole('button', { name: 'Users' })).toBeVisible();
9     await expect(page.getByRole('button', { name: 'Export' })).toBeVisible();
10 })
```

## 10 Fazit und Ausblick

### 10.1 Zusammenfassung

In dieser Diplomarbeit wurde ein Timing-System für die ZEC entwickelt. Das System bietet eine REST-API-Schnittstelle, die es ermöglicht, Versuche in die Datenbank einzutragen, welche auch automatisch bewertet werden. Zusätzlich können die Ergebnisse über die API abgerufen werden und alle wichtigen Informationen können einfach über die Weboberfläche eingesehen und bearbeitet werden.

Der Microservice-Ansatz hat sich als passend für das Projekt erwiesen, da er eine klare Trennung der Verantwortlichkeiten ermöglicht und die Wartbarkeit des Systems verbessert. Die Verwendung von Docker-Containern war ebenfalls eine richtige Entscheidung, da so die Bereitstellung erleichtert wurde und eine Unabhängigkeit von der Infrastruktur gewährleistet werden konnte. Die Verwendung einer zentralen Datenbank hätte die Entwicklung erleichtert, allerdings hätte dies die Eigenständigkeit der Services massiv eingeschränkt, weshalb die Entscheidung für einzelne Datenbanken pro Service auch hier die richtige war.

### 10.2 Zukunftsausblick

Zusätzlich zu den offensichtlichen Erweiterungen, wie der Fertigstellung der noch nicht implementierten Features, etwa der Desktop-App zur Verbindung mit der Lichtschranke per MQTT, der Kubernetes-Integration und den CI/CD-Pipelines, kamen während der Entwicklung auch andere Ideen auf, die in Zukunft umgesetzt werden könnten.

- **SD-Karten-Schutz:** Derzeit ist kein Schutz für die SD-Karten der Raspberry Pis implementiert. Eine Implementierung von Caching im Backend oder Frontend wäre notwendig, sollte der Server auf einem Raspberry Pi laufen.
- **Anbindung einer Bluetooth-Datenübertragung:** Im derzeitigen System muss der Zeitnehmer von den Sensoren ablesen, wie viel Energie verbraucht wurde. Eine Anbindung der Bluetooth-Datenübertragung könnte es ermöglichen, dass die Sensoren die Daten an die Desktop-App übermitteln.

## 11 Anhang

| Datum      | Tätigkeit                                                                                         | Zeitaufwand<br>in h |
|------------|---------------------------------------------------------------------------------------------------|---------------------|
| 11.04.2025 | Initiale Besprechung und Analyse des aktuellen Projektstands                                      | 1                   |
| 30.06.2025 | Analyse des bestehenden Repositories, Anforderungsbesprechung sowie Beginn des Diplomarbeitstrags | 3                   |
| 12.07.2025 | Weiterentwicklung des Diplomarbeitstrags                                                          | 2                   |
| 18.07.2025 | Finalisierung der ersten Version des Diplomarbeitstrags                                           | 2                   |
| 30.07.2025 | Fachlicher Informationsaustausch                                                                  | 1                   |
| 15.08.2025 | Einarbeitung in Authentifizierungsserver sowie Erstellung eines Grundkonzepts                     | 2                   |
| 31.08.2025 | Ausarbeitung des Konzepts und Erstellung eines Basisdesigns                                       | 3                   |
| 07.09.2025 | Entwicklung einer grundlegenden Seitenstruktur                                                    | 2                   |
| 09.09.2025 | Erstellung von Besprechungsprotokollen                                                            | 1.5                 |
| 10.09.2025 | Statusbesprechung zum Projektfortschritt                                                          | 0.5                 |
| 25.09.2025 | Abstimmung zum Pflichtenheft                                                                      | 0.5                 |
| 27.09.2025 | Überarbeitung des Diplomarbeitstrags                                                              | 2                   |
| 08.10.2025 | Datenbankmodellierung                                                                             | 1.5                 |
| 15.10.2025 | Vorbereitung von Besprechungsunterlagen                                                           | 0.5                 |
| 29.10.2025 | Einarbeitung in Keycloak sowie initiale Einrichtung                                               | 1                   |
| 30.10.2025 | Implementierung des Keycloak-Services                                                             | 3                   |
| 31.10.2025 | Implementierung der Authentifizierung gegen Keycloak-Datenbank                                    | 4                   |
| 02.11.2025 | Fehleranalyse und Behebung von Authentifizierungsproblemen in Docker-Umgebung                     | 4                   |
| 07.11.2025 | Projektstatusbesprechung                                                                          | 0.5                 |
| 10.11.2025 | Umsetzung eines ersten funktionalen Login Prototypen                                              | 3.5                 |
| 11.11.2025 | Refactoring und Bereinigung der Authentifizierungslogik                                           | 3                   |

| Datum      | Tätigkeit                                                                                                                   | Zeitaufwand<br>in h |
|------------|-----------------------------------------------------------------------------------------------------------------------------|---------------------|
| 12.11.2025 | Überarbeitung des Konzepts sowie Anpassung von Datenbank und Modellen                                                       | 5                   |
| 14.11.2025 | Interne Abstimmung zum Datenbankschema                                                                                      | 0.5                 |
| 19.11.2025 | Finalisierung der Datenmodelle, Vorbereitung des Frontends sowie Statusbesprechung inkl. Protokollierung                    | 3                   |
| 26.11.2025 | Implementierung von Änderungen an Datenbank und Modellen                                                                    | 1.5                 |
| 03.12.2025 | Entwicklung von CRUD-Funktionalitäten und API-Endpunkten                                                                    | 4                   |
| 06.12.2025 | Integration und Fehlerbehebung bei Mapper- und Keycloak-Komponenten                                                         | 4                   |
| 08.12.2025 | Vorbereitung von API-Services                                                                                               | 3                   |
| 09.12.2025 | Finalisierung des User-Service                                                                                              | 4                   |
| 13.12.2025 | Vorbereitung der Zwischenpräsentation                                                                                       | 3                   |
| 16.12.2025 | Implementierung des Score-Services                                                                                          | 4                   |
| 20.12.2025 | Erstellung von Besprechungsprotokollen                                                                                      | 2                   |
| 22.12.2025 | Überarbeitung des Frontends und Implementierung einer Sidebar                                                               | 5                   |
| 23.12.2025 | Implementierung der Leaderboard-Abfrage                                                                                     | 2                   |
| 24.12.2025 | Überarbeitung des User-Service                                                                                              | 2                   |
| 25.12.2025 | Weiterentwicklung des User-Service, Strukturierung des Projekts sowie Recherche zu API-Gateway und Microservice-Architektur | 4                   |
| 26.12.2025 | Umsetzung von Microservice-Architekturen                                                                                    | 5                   |
| 27.12.2025 | Entwicklung des Score-Microservices                                                                                         | 3                   |
| 28.12.2025 | Weiterentwicklung des Score-Microservices                                                                                   | 4                   |
| 29.12.2025 | Einrichtung eines grundlegenden API-Gateways                                                                                | 3                   |
| 30.12.2025 | Implementierung der Authentifizierung im API-Gateway                                                                        | 3.5                 |
| 31.12.2025 | Erstellung der API-Dokumentation                                                                                            | 3                   |
| 04.01.2026 | Refactoring des Frontends                                                                                                   | 2                   |

| Datum      | Tätigkeit                                                                                      | Zeitaufwand<br>in h |
|------------|------------------------------------------------------------------------------------------------|---------------------|
| 05.01.2026 | Implementierung von Fehlerbehandlung im Backend                                                | 3                   |
| 06.01.2026 | Durchführung von Tests                                                                         | 4                   |
| 09.01.2026 | Entwicklung der Authentifizierungs-API, Refactoring des Backends sowie Tests und Dokumentation | 6.5                 |
| 10.01.2026 | Implementierung von Authentifizierung auf Gateway-Ebene                                        | 2.5                 |
| 11.01.2026 | Refactoring der API und Authentifizierung sowie Erweiterung um Teamkategorien                  | 3.5                 |
| 12.01.2026 | Durchführung von API-Tests                                                                     | 1                   |
| 13.01.2026 | Implementierung von Validierungslogik und Begrenzung von Versuchen                             | 2.5                 |
| 17.01.2026 | Überarbeitung der Penalty-Logik                                                                | 4                   |
| 18.01.2026 | Fehlerbehebungen und Anpassungen in Frontend und Backend                                       | 2.5                 |
| 19.01.2026 | Implementierung eines Frontend-Footers                                                         | 1                   |
| 20.01.2026 | Erweiterung um Login- und Account-Funktionalitäten                                             | 1                   |
| 23.01.2026 | Implementierung rollenbasierter Zugriffskontrolle im Frontend mit Backend-Anbindung            | 2                   |
| 24.01.2026 | Fehleranalyse (CORS) sowie Erweiterung des User-Bereichs                                       | 4                   |
| 25.01.2026 | Anpassung von Pydantic-Schemas und Nginx-Konfiguration                                         | 1                   |
| 30.01.2026 | Weiterführende Fehleranalyse im Bereich CORS                                                   | 2                   |
| 02.02.2026 | Durchführung von Tests für Score-Funktionalität                                                | 2                   |
| 06.02.2026 | Erweiterte Tests und Debugging im Score- und Attempt-Bereich                                   | 4                   |
| 07.02.2026 | Optimierung und Bereinigung von Warnungen im Testsystem                                        | 2                   |
| 08.02.2026 | Erweiterung des Frontends um Attempt-Ansicht und UI-Optimierung                                | 2                   |

| Datum         | Tätigkeit                                                                                      | Zeitaufwand<br>in h |
|---------------|------------------------------------------------------------------------------------------------|---------------------|
| 11.02.2026    | Fehleranalyse und Verbesserung der Stabilität in Frontend und Backend                          | 3                   |
| 12.02.2026    | Projektbesprechung, Systemintegration sowie Erstellung der Dokumentationsstruktur              | 1.5                 |
| 13.02.2026    | UI-Optimierungen im Frontend                                                                   | 1                   |
| 14.02.2026    | Initialisierung der Projektdokumentation im Repository                                         | 1                   |
| 15.02.2026    | Erweiterung der API-Dokumentation und kleinere Anpassungen im Auth-Service                     | 3                   |
| 16.02.2026    | Implementierung von Testdaten sowie Überarbeitung der Score-Berechnung                         | 3                   |
| 17.02.2026    | Implementierung der Benutzerzuweisung zu Teams                                                 | 2                   |
| 20.02.2026    | Überarbeitung des User-Service                                                                 | 2                   |
| 22.02.2026    | Anpassung von Teamlead-Berechtigungen sowie Implementierung von Session-Management im Frontend | 3                   |
| 23.02.2026    | Implementierung der Exportlogik                                                                | 2                   |
| 24.02.2026    | Durchführung von API-Tests                                                                     | 1                   |
| 25.02.2026    | Durchführung von API-Tests                                                                     | 1                   |
| 26.02.2026    | Durchführung von API-Tests                                                                     | 1                   |
| 03.03.2026    | Überarbeitung der Dokumentationsstruktur                                                       | 2                   |
| 08.03.2026    | Durchführung von Frontend-Tests                                                                | 3                   |
| 01.04.2026    | Diplomarbeitsdokumentation                                                                     | 8                   |
| 02.04.2026    | Diplomarbeitsdokumentation                                                                     | 8                   |
| 03.04.2026    | Diplomarbeitsdokumentation                                                                     | 8                   |
| 04.04.2026    | Diplomarbeitsdokumentation                                                                     | 8                   |
| <b>Gesamt</b> |                                                                                                | <b>224</b>          |

## 12 Verzeichnisse

### 12.1 Abbildungsverzeichnis

|   |                                                     |    |
|---|-----------------------------------------------------|----|
| 1 | Docker Logo . . . . .                               | 11 |
| 2 | Systemarchitektur . . . . .                         | 17 |
| 3 | Datenbankschema des ZEC-Timing-Systems . . . . .    | 18 |
| 4 | Leaderboard-Tab für Viewer . . . . .                | 26 |
| 5 | Teams-Tab für Teamleiter . . . . .                  | 27 |
| 6 | Teams-Tab für Administrator . . . . .               | 27 |
| 7 | Sequenzdiagramm: Login-Ablauf . . . . .             | 34 |
| 8 | Sequenzdiagramm: Authentifizierte Anfrage . . . . . | 35 |
| 9 | Sequenzdiagramm: Token-Refresh . . . . .            | 36 |

## 12.2 Tabellenverzeichnis

|   |                                              |    |
|---|----------------------------------------------|----|
| 1 | Datenbankschema: User Service . . . . .      | 20 |
| 2 | Datenbankschema: Teams . . . . .             | 21 |
| 3 | Datenbankschema: Drivers . . . . .           | 21 |
| 4 | Datenbankschema: Challenge Service . . . . . | 22 |
| 5 | Datenbankschema: Attempt Service . . . . .   | 23 |
| 6 | Datenbankschema: Scores . . . . .            | 24 |
| 7 | Datenbankschema: Penalties . . . . .         | 24 |
| 8 | Datenbankschema: Penalty Types . . . . .     | 24 |



## 12.3 Abkürzungsverzeichnis

**API** Application Programming Interface. 9, 10, 11, 12, 13, 14, 16, 18, 19, 20, 21, 25, 29, 30, 41, 42

**CI/CD** Continuous Integration / Continuous Deployment. 4, 5, 42

**CRUD** Create, Read, Update, Delete. 31

**DA** Diplomarbeit. 6, 7, 12, 13, 14, 15, 16

**JWT** JSON Web Token. 14, 15, 19

**SQL** Structured Query Language. 15

**ZEC** Zero Emission Challenge. 17, 22, 25, 42

## 12.4 Literaturverzeichnis

- [1] Amazon Web Services. *What is Containerization?* URL: <https://aws.amazon.com/what-is/containerization/> (besucht am 02. 04. 2026).
- [2] Docker. *Docker overview*. URL: <https://docs.docker.com/get-started/docker-overview/> (besucht am 02. 04. 2026).
- [3] Docker. *Dockerfile reference*. URL: <https://docs.docker.com/build/concepts/dockerfile/> (besucht am 02. 04. 2026).
- [4] Docker. *Docker Compose application model*. URL: <https://docs.docker.com/compose/intro/compose-application-model/> (besucht am 02. 04. 2026).
- [5] Amazon Web Services. *What are Microservices?* URL: <https://aws.amazon.com/microservices/> (besucht am 02. 04. 2026).
- [6] GitHub. *GitHub*. URL: <https://github.com/> (besucht am 02. 04. 2026).
- [7] F5, Inc. *nginx*. URL: <https://nginx.org/> (besucht am 02. 04. 2026).
- [8] The PostgreSQL Global Development Group. *PostgreSQL: The World's Most Advanced Open Source Relational Database*. URL: <https://www.postgresql.org/> (besucht am 02. 04. 2026).
- [9] Red Hat, Inc. *Open Source Identity and Access Management*. URL: <https://www.keycloak.org/> (besucht am 02. 04. 2026).
- [10] SQLAlchemy Authors. *The Python SQL Toolkit and Object Relational Mapper*. URL: <https://www.sqlalchemy.org/> (besucht am 02. 04. 2026).
- [11] Sebastián Ramírez. *FastAPI*. URL: <https://fastapi.tiangolo.com/> (besucht am 02. 04. 2026).
- [12] JetBrains. *Which Is the Best Python Web Framework: Django, Flask, or FastAPI?* URL: <https://blog.jetbrains.com/pycharm/2025/02/django-flask-fastapi/> (besucht am 02. 04. 2026).
- [13] Vercel. *Next.js*. URL: <https://nextjs.org/> (besucht am 02. 04. 2026).
- [14] shadcn. *The Foundation for your Design System*. URL: <https://ui.shadcn.com/> (besucht am 02. 04. 2026).
- [15] Python Software Foundation. *Python HTTP for Humans*. URL: <https://pypi.org/project/requests/> (besucht am 03. 04. 2026).
- [16] pytest. *pytest: helps you write better programs*. URL: <https://docs.pytest.org/en/stable/> (besucht am 02. 04. 2026).

- [17] Microsoft Corporation. *Playwright enables reliable web automation for testing, scripting, and AI agents*. URL: <https://playwright.dev/> (besucht am 03.04.2026).
- [18] pytest-cov. *Pytest plugin for measuring coverage*. URL: <https://pypi.org/project/pytest-cov/> (besucht am 02.04.2026).
- [19] SQLite Consortium. *SQLite*. URL: <https://www.sqlite.org/> (besucht am 03.04.2026).
- [20] pytest. *About fixtures*. URL: <https://docs.pytest.org/en/stable/fixture.html> (besucht am 03.04.2026).
- [21] Python Software Foundation. *unittest.mock — mock object library*. URL: <https://docs.python.org/3/library/unittest.mock.html> (besucht am 03.04.2026).
- [22] pytest. *monkeypatch*. URL: <https://docs.pytest.org/en/stable/reference.html#pytest.MonkeyPatch> (besucht am 03.04.2026).
- [23] Sebastián Ramírez. *Test Client*. URL: <https://fastapi.tiangolo.com/reference/testclient/> (besucht am 03.04.2026).